

[IS216] Extra Exercises - Vue Components

Objectives

- Gain a solid understanding of reactive programming concepts.
- Learn to leverage Vue framework features for building efficient web applications.
- Practice applying Vue directives to create dynamic and responsive web pages.

Instructions

- Questions without an asterisk are easy.
- Questions marked with * are moderately challenging.
- Questions marked with ** are challenging.
- Questions marked with *** are very challenging.

Download

- **Resources:** Click [here](#)
- **Sample Solutions:** Click [here](#)

Contents

Question 1 - Open Weather (*).....	2
Question 2 - Weather and Temperature (**).....	4
Question 3 - Two Lists (**).....	8
Question 4 - Login (**).....	11
Question 5 - Mart (***).....	13

Note:

- If you notice any mistakes or errors in the questions, please email your instructors with a description of the issue. We will address it as soon as possible.
- For the best viewing experience, please use Microsoft Office 365 Online.

Question 1 - Open Weather (*)

Background:

You are tasked with developing a weather station web application that displays real-time weather information for four different cities. The weather data should be presented in a clear and organized tabular format (see Figure q1a).

Requirements:

- When the application first loads (see Figure q1a):
 - It should display the weather information for four different cities using a Vue component named <Weather>.
- The <Weather> component must:
 - Accept a prop called city.
 - Perform an API call to <https://api.openweathermap.org/data/2.5/weather> to retrieve weather data (description, temperature, humidity and wind speed) for the given city.
 - For more details, refer to the OpenWeatherMap API documentation: <https://openweathermap.org/current>
 - Convert the temperature value from Kelvin to degrees Celsius.
 - Construct and display a descriptive string containing the retrieved weather information.

Instructions:

1. Navigate to the q1 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. Navigate to the q1/src/components directory.
5. Open the Weather.vue file and add or modify the code to fulfill the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 500px.
- You need to create an account at <https://openweathermap.org> and generate an API key in order to successfully retrieve the weather data.

When the application first loads.



Figure q1a

Question 2 - Weather and Temperature (**)

Background:

You are tasked with developing a weather station web application. The application should display real-time weather information for Singapore which include a navigation menu that allows users to view various types of forecast data, such as temperature, 2-hourly, 24-hour, and 4-day forecasts (see Figure q2a).

Requirements:

- When the application first loads (see Figure q2a):
 - A top menu should be displayed, consisting of four tabs for navigating between different types of forecast data. By default, the Singapore temperature forecast should be shown.
 - The top menu and each of the four forecast types should be implemented using individual Vue components named `<TopMenu>`, `<Temperature>`, `<Forecast2Hourly>`, `<Forecast24Hour>`, and `<Forecast4Days>`.
- The `<TopMenu>` component must:
 - Display a responsive bootstrap navigation bar.
 - Accept a `menuitems` prop (an array of strings) representing the list of top menu labels to be displayed.
 - Accept an `activeitem` prop that sets the initially highlighted (active) menu item.
 - Loop through all values in `menuitems` using `v-for`, creating a `<li>` for each item in the top menu.
 - Set the clicked item as the current active item and emit a click event with the clicked item as the event payload.
- The `<Temperature>` component must:
 - Display a table listing consists of Weather stations of different locations and their corresponding temperature values.
 - The readings are fetched from the NEA Air Temperature API:
 - <https://api-open.data.gov.sg/v2/real-time/api/air-temperature>
 - Use the `created()` lifecycle hook to automatically trigger data retrieval when the component is initialized.
 - Provide a "Refresh" button that triggers the data fetch again on user click.
 - Show the current date and time (in UTC) when the data was retrieved.
 - If the API request fails (e.g., network or server issue), show an error message using `v-if`.
- The `<Forecast2Hourly>` component must:
 - Display a table where:
 - The first row (header) shows the start times timeslot.
 - Each subsequent row shows forecast data for a specific area.
 - Forecasts that span multiple timeslots are shown in merged `<td>` cells.
 - Fetch the 2-hour weather forecast data from the data.gov.sg Weather API link given below when the component is created. The request must include a date parameter, derived from the current date.
 - <https://api-open.data.gov.sg/v2/real-time/api/two-hr-forecast>

Commented [TF1]: Hi @Alvin TAY, I assume you are taking care of this. Pardon me if I get the wrong person.

This TopMenu exercise for emitting event and displaying made sense last time because Vue router was not covered.

Changing TopMenu to make use of Vue router may align with the syllabus better.

Commented [LT2R1]: Hi @THIANG Lay Fod, yes, I'm the right person! I've modified Q2 to use Vue Router instead.

Commented [TF3]: There is a v2 Doc: https://data.gov.sg/datasets/d_3f9e064e25005b0e42969944ccaf2e7a/view

API: <https://api-open.data.gov.sg/v2/real-time/api/two-hr-forecast> slightly more data.

v1 is still working.

Commented [LT4R3]: I've switched to using the v2 API in Q2 and solution. Please help to accept the deletion of the old v1 URL. Thank you.

- The <Forecast24Hour> component must:
 - Display the overall weather forecast summary from the most recent forecast data.
 - Display a table showing key forecast parameters:
 - Relative Humidity (low and high values)
 - Temperature (low and high values)
 - Wind Speed (low and high values) including wind direction in the column header
 - Display another table showing forecast information for each period, including:
 - Start time and End time
 - Weather forecasts for each region: North, South, East, West, Central
 - Fetch the latest 24-hour weather forecast data from the data.gov.sg Weather API link given below when the component is created. The request must include a date parameter, derived from the current date.
 - <https://api-open.data.gov.sg/v2/real-time/api/twenty-four-hr-forecast>
- The <Forecast4Days> component must:
 - Display a table showing forecast data for the next 4 days. Each row must include:
 - Date
 - Forecast (e.g., Showers, Thunderstorms)
 - Temperature Range: Low and High (in °C)
 - Relative Humidity Range: Low and High (in %)
 - Wind Details:
 - Direction (e.g., N, NE, SW)
 - Wind Speed Range: Low and High (in km/h)
 - Fetch the 4-day weather forecast from the data.gov.sg Weather API link given below when the component is created. The request must include a date parameter, derived from the current date.
 - <https://api-open.data.gov.sg/v2/real-time/api/four-day-outlook>

Instructions:

1. Navigate to the q2 directory in your terminal.
2. Run the command “pnpm install” to install the required dependencies.
3. Run the command “pnpm dev” to run the app in your development server.
4. Navigate to the q2/src/components directory.
5. Open the TopMenu.vue, Temperature.vue, Forecast2Hourly.vue, Forecast24Hour.vue and Forecast4Days.vue files and add or modify the code to fulfill the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 800px.

Commented [TF5]: There is a v2 Doc:
https://data.gov.sg/datasets/d_ce2eb1e307bda31993c533285834ef2b/view

API: <https://api-open.data.gov.sg/v2/real-time/api/twenty-four-hr-forecast>

Different JSON structure from v1
 Otherwise, similar data

Commented [LT6R5]: I've switched to using the v2 API in Q2 and solution. Please help to accept the deletion of the old v1 URL. Thank you.

Commented [TF7]: There is a v2 Doc:
https://data.gov.sg/datasets/d_f131f6e343bf8168e4057a04c4326a0a/view

API: api-open.data.gov.sg/v2/real-time/api/four-day-outlook

The API is NOT working now.

Last night, it was different json structure but similar data.

Commented [LT8R7]: The v1 API works at times. I've switched to using the v2 API in Q2 and solution. Please help to accept the deletion of the old v1 URL. Thank you.

When the application first loads.

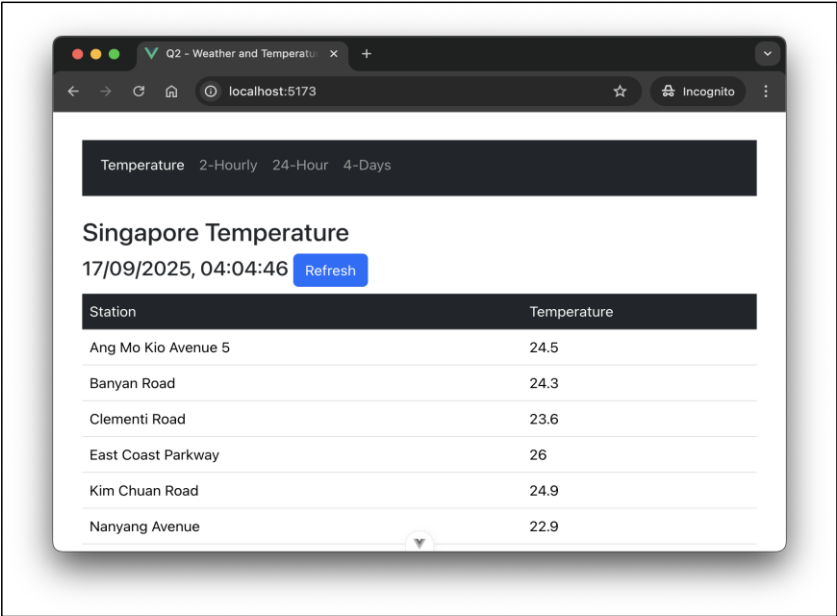


Figure q2a

When the “2-Hourly” menu tab is clicked.

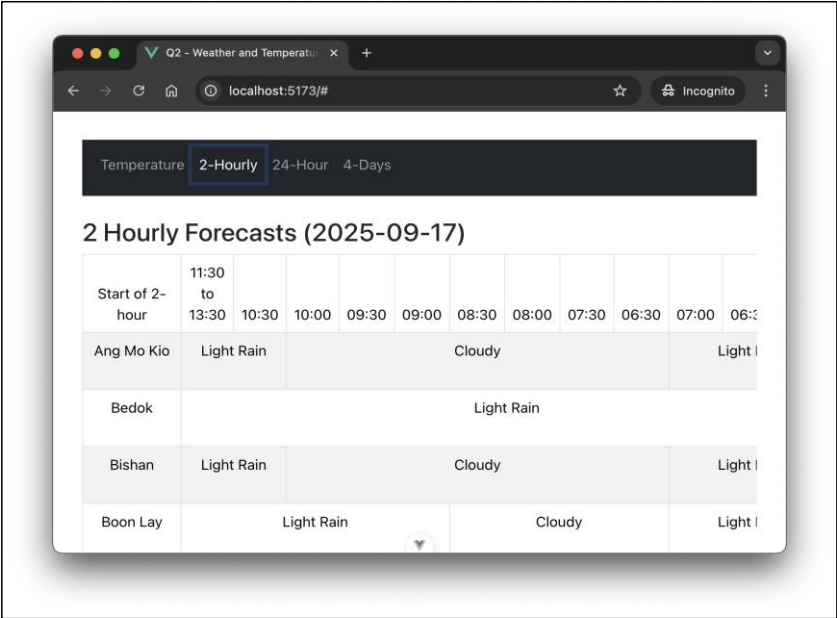


Figure q2b

When the “24-Hour” menu tab is clicked.

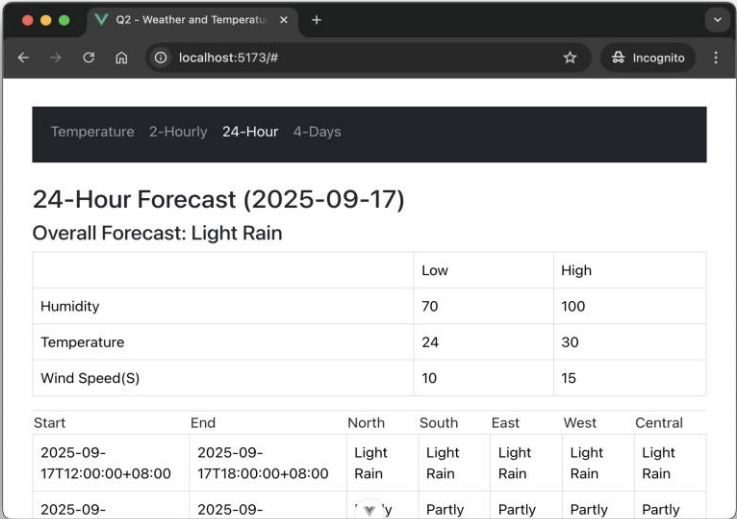


Figure q2c

When the “4-Days” menu tab is clicked.

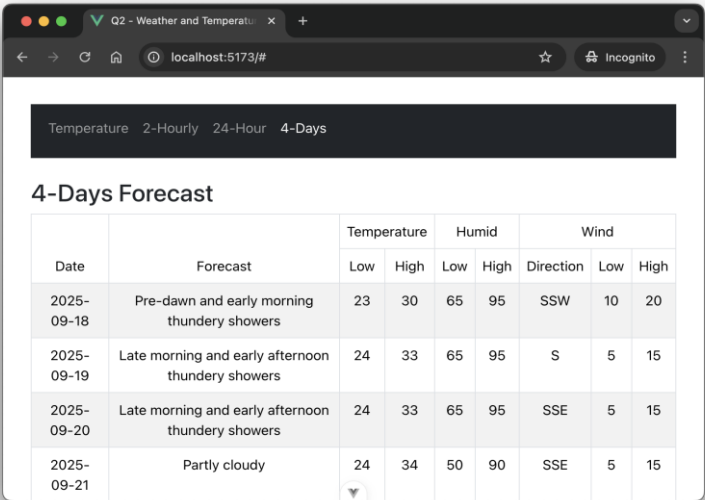


Figure q2d

Question 3 - Two Lists (**)

Background:

You are tasked with developing a "Two Lists" web application. The application should allow users to easily move items between two lists, enabling intuitive classification and preference management such as sorting items into likes versus dislikes and categorizing contacts as friends versus acquaintances.

Requirements:

- When the application first loads (see Figure q3a):
 - It should display the "Fruits" and "People" sections, each containing two lists rendered using the Vue component `<TwoLists>`.
- The `<TwoLists>` component must:
 - Receive the following props:
 - `size`: Number of visible rows in the list.
 - `width`: The width of the list area (fixed at 500px).
 - `left-title`: Title label for the left-hand list.
 - `right-title`: Title label for the right-hand list.
 - `left-list`: The array to be displayed on the left-hand side.
 - `right-list`: The array to be displayed on the right-hand side.
 - Dynamically render the items within the lists based on the provided arrays.
 - Display two buttons between the lists:
 - A `>>` button for moving selected items from the left list to the right list.
 - A `<<` button for moving selected items from the right list to the left list.
 - Perform the following transfer behaviour:
 - When the `>>` button is clicked (see Figure q3b):
 - All selected items in the left list shall be removed from left-list and appended to right-list.
 - The selection shall be cleared after the transfer.
 - When the `<<` button is clicked (see Figure q3c):
 - All selected items in the right list shall be removed from right-list and appended to left-list.
 - The selection shall be cleared after the transfer.

Instructions:

1. Navigate to the q3 directory in your terminal.
2. Run the command `pnpm install` to install the required dependencies.
3. Run the command `pnpm dev` to run the app in your development server.
4. Navigate to the `q3/src/components` directory.
5. Open the `TwoList.vue` file and add or modify the code to fulfill the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 600px.

When the application first loads.

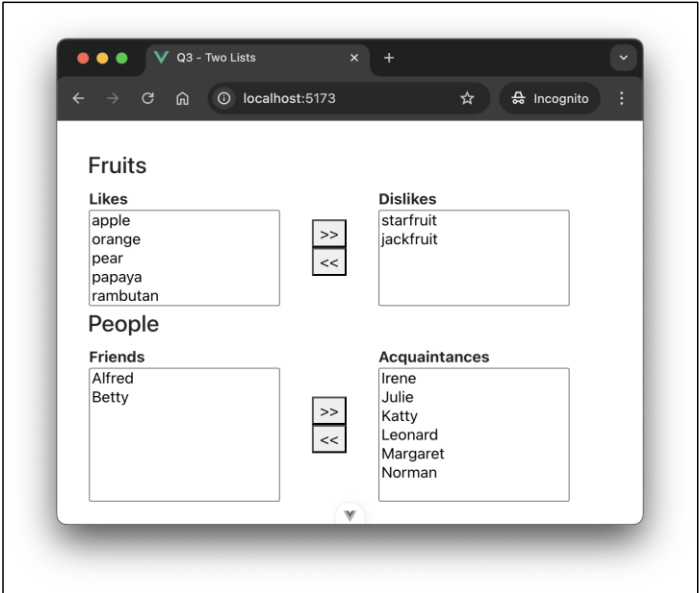


Figure q3a

When "apple" and "orange" are selected and the ">>" button is clicked.

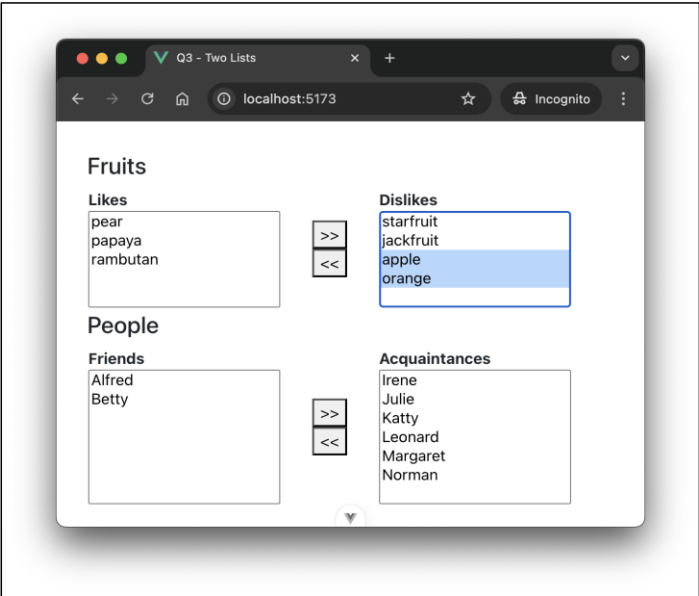


Figure q3b

When "Katty" and "Leonard" are selected and the "<<" button is clicked.

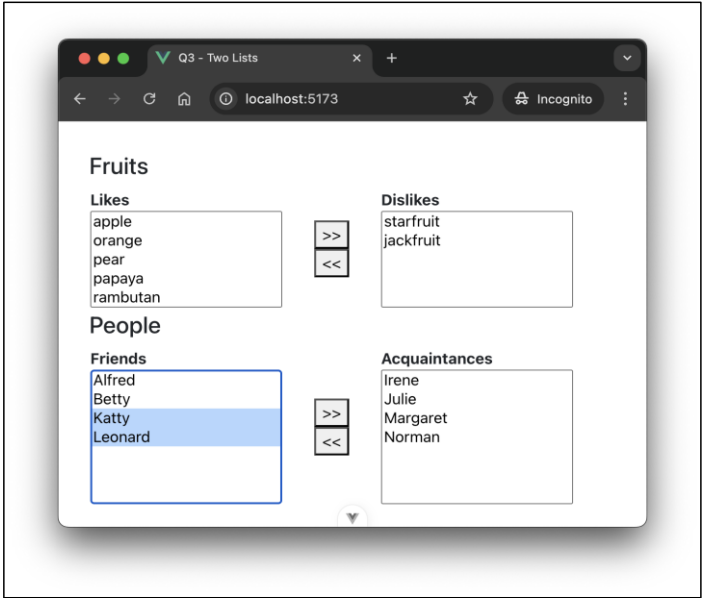


Figure q3c

Question 4 - Login (**)

Background:

You are tasked with enhancing the server-side authentication for a web application. The application should feature a login interface where users can sign in using a predefined user ID and password. Upon successful authentication, the user should be greeted with a welcome message and presented with a logout option. If authentication fails, an appropriate error message should be displayed.

Requirements:

- When the application first loads (see Figure q4a):
 - It should display a login form with user ID and password input field as well as a Login button using a Vue component <MyLogin>
- The <MyLogin> component must:
 - Render a login form consisting of:
 - A text input field for entering the User ID.
 - A password input field for entering the Password.
 - A login button to initiate the authentication process.
 - Upon clicking the Login button:
 - The component shall send an HTTP POST request using Axios to:
 - <http://localhost:3000/login>
 - If the server response indicates failure (see Figure q4b):
 - The component shall display the message "Invalid user ID or password [HTTP Status: 401 Unauthorized]" below the form in red text.
 - If the response from the server indicates success (see Figure q4c):
 - The password field should be cleared for security purposes.
 - An event should be emitted to the parent component, including the user's name, to be displayed on the welcome page.
 - If there is no response from the server as it is terminated or connection is lost (see Figure q4d):
 - The component shall display the message "Unable to authenticate. [Error Code: ERR_NETWORK]" below the form in red text.

Instructions:

1. Navigate to the q4/server/ directory in your terminal.
2. Run the command "pnpm install" to install the required dependencies.
3. Run the command "pnpm dev" to run the Node.js app (backend) in your development server.
4. Open a new terminal and navigate to the q4 directory.
5. Run the command "pnpm install" to install the required dependencies.
6. Run the command "pnpm dev" to run the Vue app (frontend) in your development server.
7. Navigate to the q4/src/components directory.
8. Open the MyLogin.vue file and add or modify the code to fulfill the requirements described above.

Commented [TF9]: Hi @Alvin TAY,

Hope I get the right person.

Suggest to change authenticate.js to send HTTP response 401 when authentication fails.

```
...
// Authenticate a user
function doLogin(userid, pwd) {
  // Create account manager
  const mgr = new AccountMgr();

  // Retrieve account by userid
  const account = mgr.get(userid);

  // Check that the account exists and password
  matches
  if (account && account.getPassword() === pwd) {
    return {
      status: true,
      userid: userid,
      name: account.getName()
    };
  }

  return false;
}

// Handle login requests
app.post("/login", (req, res) => {
  // Default response
  let results = false;

  // Read userid and password from request body
  const { userid, pwd } = req.body;

  results = doLogin(userid, pwd, results);

  // Return the result as JSON
  if (results === false)
    res.status(401).json({
      status: false,
    });
  else res.json(results);
});
...
```

Commented [LT10R9]: I've updated Q4 to return the error message with the HTTP/error code.

Notes:

- The recommended browser width for viewing all the figures is 500px.

When the application first loads.

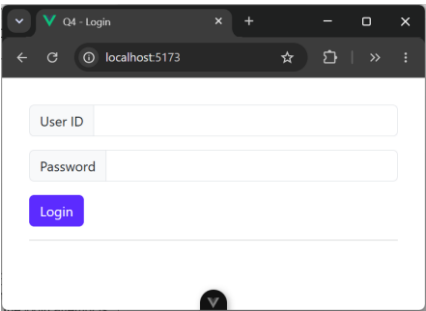


Figure q4a

When the user enters the user ID "apple.2026" and password "12345", but the login attempt fails.

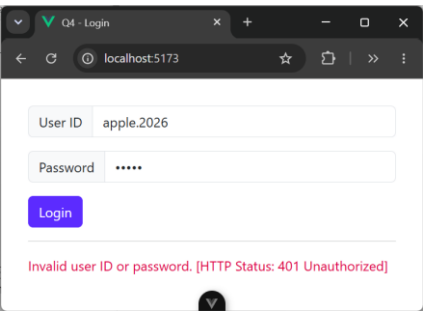


Figure q4b

When the user enters the user ID "apple.2026" and password "apple.2026.pwd", and the login attempt is successful.

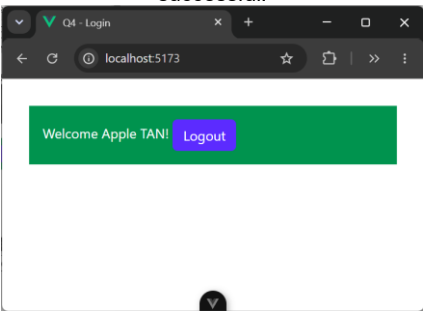


Figure q4c

When the Node.js app (backend) development server is terminated and the user enters the user ID "apple.2026" and password "apple.2026.pwd", but the login attempt fails.

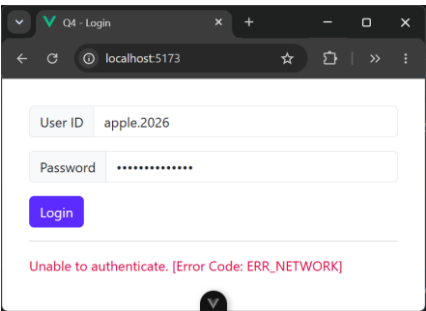


Figure q4d

Question 5 - Mart (***)

Background:

You are tasked with developing a simple shopping cart web application that enables users to browse items by category, select desired quantities, add them to a shopping cart, and proceed to checkout. The application should seamlessly interact with a backend server via Axios to retrieve available item categories, fetch detailed information for items within the selected category, and process checkout transactions efficiently.

Requirements:

Part A

- When the application first loads (see Figure q5a):
 - It should display the following:
 - A Categories drop-down menu containing three categories: fruit, meat, and drink.
 - A table displaying items based on the selected category.
 - A Cart table showing items that have been added. The Cart table should be empty by default.
 - An “Add to Cart” button to add selected items along with their specified quantities to the cart.
 - The display for the Categories drop-down menu, the item browsing table and the “Add to Cart” button is rendered using the Vue component `<ItemsBrowser>` which performs the following functions:
 - Includes a “button_label” prop that specifies the text to be displayed on its button.
 - Sends an HTTP GET request using Axios to the link below upon creation and uses the response to populate the categories drop-down menu. The first category in the menu is automatically selected and used to fetch its associated items.
 - <http://localhost:3000/categories>
 - Sends an HTTP GET request using Axios to the link below with the user-selected category as a query parameter. The response is used to populate the items table with item name, price, and a quantity input field. Each item's quantity is initially set to 0.
 - <http://localhost:3000/items>
 - When the “Add to Cart” button is clicked, all items with a quantity greater than 0 are collected into a temporary array. This array is emitted as a payload via a click event to the parent component, and the quantities of the added items are then reset to 0.

Part B

- When the “Add to Cart” button is clicked (see Figure q5b):
 - The display of the application should update as follows:
 - The Cart table displays the selected items along with their quantities and computes the total amount to be paid.
 - A Checkout button appears, allowing the user to proceed with the purchase.

Commented [TF11]: Either add a reminder to kill q4 server or change the port to avoid port clashes

Commented [LT12R11]: I've added a point in the instructions for Q5 to terminate the previous question's server, if it was not terminated already.

- The display for the Cart table, the total amount to be paid, and the “Checkout” button is rendered using the Vue component `<ShoppingCart>` which performs the following functions:
 - Includes a “button_label” prop that specifies the text to be displayed on the button.
 - Accept a “cart_items” prop, containing an array of items added to the cart.
 - Uses a computed property to calculate the total price of all cart items.
 - It emits a click event when the “Checkout” button is pressed, triggering the checkout method in the parent component, which in turn displays a new page (see Figure q5c).

Instructions:

1. In the terminal for the previous question’s Node.js app (backend) development server, press `Ctrl + C` to terminate it, if it is not already terminated.
2. Navigate to the `q5/server/` directory in your terminal.
3. Run the command “`pnpm install`” to install the required dependencies.
4. Run the command “`pnpm dev`” to run the Node.js app (backend) in your development server.
5. Open a new terminal and navigate to the `q5` directory.
6. Run the command “`pnpm install`” to install the required dependencies.
7. Run the command “`pnpm dev`” to run the Vue app (frontend) in your development server.
8. Navigate to the `q5/src/components` directory.
9. Open the `ItemsBrowser.vue` and `ShoppingCart.vue` files and add or modify the code to fulfill the requirements described above.

Notes:

- The recommended browser width for viewing all the figures is 800px.

When the application first loads.

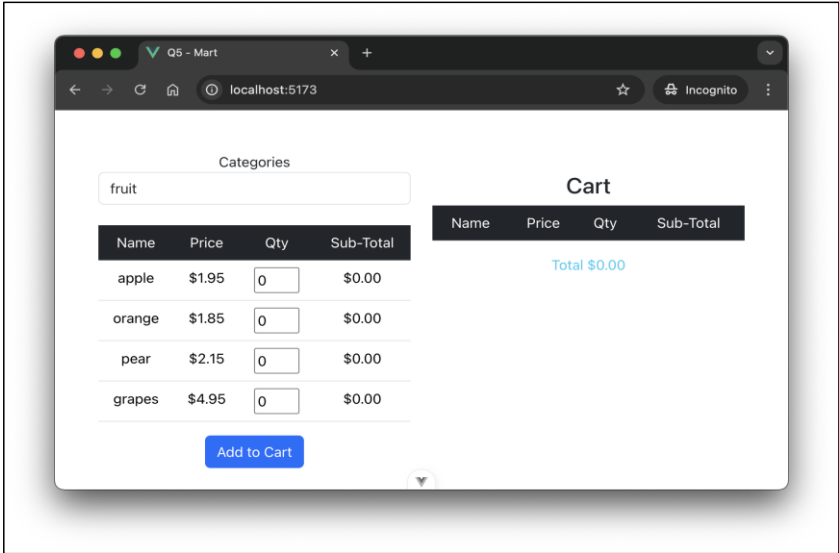


Figure q5

When the “Add to Cart” button is clicked after entering a quantity of 2 for oranges and 3 for grapes.

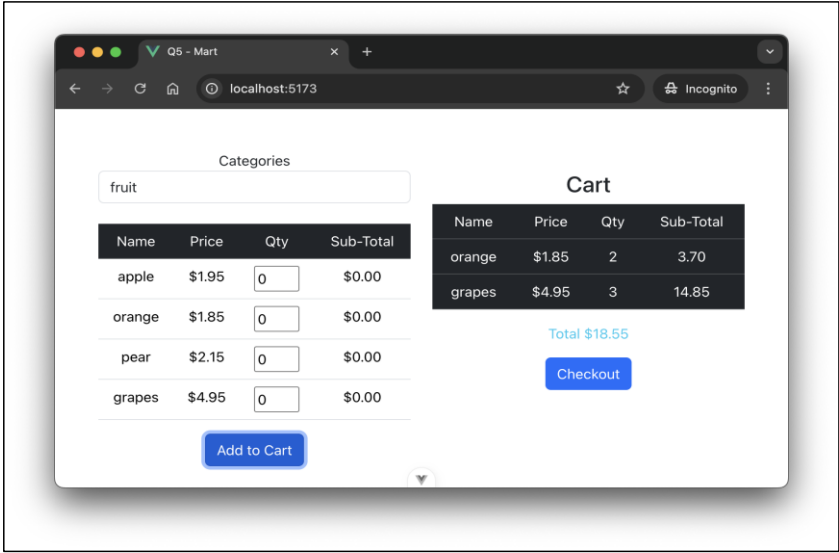


Figure q5b

When the "Checkout" button is clicked.

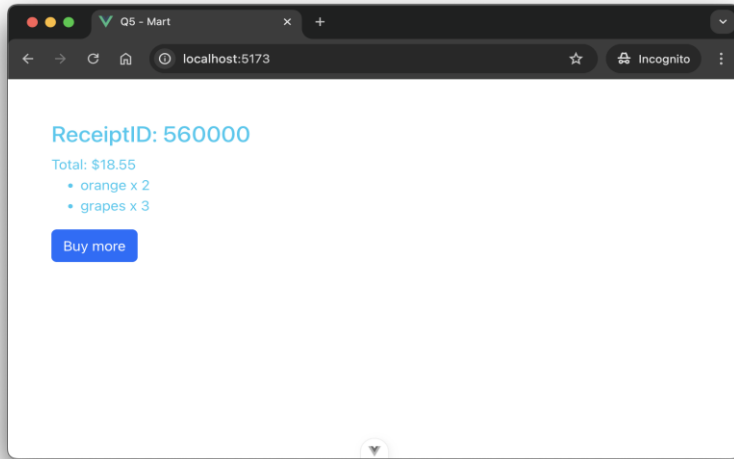


Figure q5c